DSC 102: Systems for Scalable Analytics

Programming Assignment 3: Machine Learning Competition Challenge 1

Released 26 October 2026 Due 16 November 2026, 11:59 PM Pacific

VERY IMPORTANT: There are no late days for this assignment, so plan your work accordingly. You may submit to the AI Coding Gym leaderboard as many times as you like and only your best eligible score counts, so submit early and often. You must use a local AI coding agent, such as Codex or Claude Code, and submit its original session logs. Do not download the original Kaggle data or look up existing solutions to this competition (see Section 6). DataHub GPUs are limited, and you may have to wait in a queue (see Section 5). Unless you have your own compute, start early.

1 Introduction

In this assignment you will work on a real machine learning competition from AI Coding Gym, **Ventilator Pressure Prediction**, where frontier AI agents fall short. Your goal is to outperform them by applying your own judgment, running careful experiments, and using AI assistants as collaborative tools. Your leaderboard score matters, but understanding matters more. A perceptive analysis of what worked, what did not, and why is the core of this assignment.

Challenge page:
https://aicodinggym.com/challenges/mle/ventilator-pressure-prediction

2 Problem and Dataset

You will predict the airway pressure in the respiratory circuit during each breath, given the time series of control inputs and the attributes of the lung. The data was produced by a modified open-source ventilator connected to an artificial bellows test lung. Each breath lasts about three seconds and contains 80 time steps.

The challenge download includes `train.csv` with pressure labels, `test.csv` with the inputs only, and `sample_submission.csv` with the expected output format.

Column	Description
`id`	Unique row identifier within the file
`breath_id`	Identifier shared by the 80 rows of a breath
`R`	Airway resistance in cmH_2O per L/s of airflow. Higher resistance makes it harder to move air.
`C`	Lung compliance in mL/cmH_2O. A more compliant lung expands more for a given change in pressure.
`time_step`	Timestamp within the breath
`u_in`	Inspiratory valve opening, from 0 to 100 percent
`u_out`	Expiratory valve state, closed (0) or open (1)
`pressure`	Airway pressure in cmH_2O. This is the target.

3 Evaluation and Leaderboard

Submissions are scored by mean absolute error (MAE) across the n inspiratory rows (`u_out` equal to 0), where y_i is the measured pressure and $\hat{y}_i$ is your prediction. Lower is better.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{y}_i| \,.$$

Submission format. Submit a CSV with the header `id,pressure` and one finite pressure prediction for each test `id`, including expiratory rows. Use `sample_submission.csv` to check the format.

Use the numeric score in **Check Results**. The platform may label it `dice-hausdorff-combo`, but the benchmark grader computes inspiratory MAE.

Leaderboard. These are the AI Coding Gym benchmark scores and historical reference points from the original competition. See Section 6 for course cutoffs.

AI benchmark	**MAE**	**Original competition**	**MAE**
MLEvolve	0.14237	Gold medal line	0.1119
Disarray	0.15171	Silver medal line	0.1338
AIBuildAI	1.07594	Bronze medal line	0.1364
		Median	0.1638

4 Experiment Workflow

- **Start with a baseline.** Establish a valid prediction pipeline and measure its error. Hold out whole breaths from `train.csv`, and use the same split and inspiratory MAE to compare your experiments.
- **Choose what to investigate.** Look for patterns in your validation errors and use the challenge's **AI Benchmark** panel for ideas. You might examine features, model choices, or how you train, depending on what you find.
- **Compare and submit.** Record each trial's MAE and runtime and explain what the results led you to try next. Use the leaderboard to check promising models after local validation.

See Section 7 for ways to set up an experiment loop with your coding agent.

5 Compute and Platform Setup

Use a local AI coding agent. You must complete this assignment with a local coding agent such as Codex or Claude Code. Cursor and Gemini CLI also work. A local agent can run your experiments and produce the original session logs you must submit. Open your PA3 folder in the agent and give it this handout and your AI Coding Gym profile ID. Complete sign-in and Duo yourself.

DataHub limits and tips.

- The course uses a shared pool of NVIDIA GTX 1080 Ti GPUs (11 GB). You may have to wait for a GPU, especially near the deadline. Start early if you plan to use DataHub.
- Each user can run at most 1 GPU, 8 CPUs, and 64 GB of RAM at a time.
- Containers stop after six hours by default. To request up to 12 hours, set `K8S_TIMEOUT_SECONDS` before launching. For example, put `K8S_TIMEOUT_SECONDS=43200` immediately before the remote launch command.
- A foreground container stops when you disconnect from `dsmlp-login`. Use `-b` for background runs and reconnect with `kubesh <pod-name>`.
- Save checkpoints regularly in your home directory so you can resume training.
- When finished, run `kubectl get pod` on the login server to find your pod, then free the GPU with `kubectl delete pod <pod-name>`.
- For DataHub problems, email datahub@ucsd.edu with the course ID (`DSC102_FA26_001`), the command you ran, and a screenshot.

DataHub. Have your agent run the commands below in Bash on your computer. Set your UCSD username and public key path. If you need a key, run `ssh-keygen -t ed25519`. Run `ssh-copy-id` once. See the UCSD DataHub guide for SSH and container options.

```
PA3_LOGIN=your_ucsd_username@dsmlp-login.ucsd.edu
ssh-copy-id -i ~/.ssh/id_ed25519.pub "$PA3_LOGIN"
ssh "$PA3_LOGIN" 'TERM=dumb bash -lc "/opt/launch-sh/bin/launch-scipy-ml.sh \
    -W DSC102_FA26_001 -g 1 -c 8 -m 16 -b -N pa3"'
ssh "$PA3_LOGIN" 'TERM=dumb bash -lc "kubectl exec pa3 -- nvidia-smi"'
```

To open a container shell yourself after the GPU check, run this from your computer.

```
ssh -t "$PA3_LOGIN" 'TERM=dumb bash -lc "/opt/launch-sh/bin/kubesh pa3"'
```

AI Coding Gym

Run this block in your training environment. On DataHub, start in the container's home directory. Set `GYM_USER_ID` to your AI Coding Gym profile ID.

```
GYM_USER_ID=your_profile_id
python -m pip install --user --upgrade aicodinggym-cli
export PATH="$HOME/.local/bin:$PATH"
aicodinggym mle download ventilator-pressure-prediction \
    --user-id "$GYM_USER_ID" --workspace-dir "$PWD"
cd ventilator-pressure-prediction
unzip -n data/ventilator-pressure-prediction.zip -d data
```

After downloading the data, your agent can copy a local training script to DataHub and run it in the container.

```
ssh "$PA3_LOGIN" 'TERM=dumb bash -lc "kubectl exec -i pa3 -- \
    tee ventilator-pressure-prediction/train.py >/dev/null"' < train.py
ssh "$PA3_LOGIN" 'TERM=dumb bash -ls' <<'LOGIN_NODE'
kubectl exec pa3 -- sh -c 'cd ventilator-pressure-prediction && python -u train.py'
LOGIN_NODE
```

Review `submission.csv` and submit it from the challenge directory. In a new shell, return to that directory and set `GYM_USER_ID`. This command uploads only the CSV.

```
export PATH="$HOME/.local/bin:$PATH"
aicodinggym mle submit ventilator-pressure-prediction -F submission.csv \
    --user-id "$GYM_USER_ID" --no-upload-logs --workspace-dir "$(mktemp -d)"
```

Alternative GPU resources (optional). If DataHub GPUs are busy or unavailable, you may use GPU resources from Colab. In Colab, select a GPU under **Runtime** → **Change runtime type** and run the AI Coding Gym commands in `%%bash` cells. Have your local agent prepare the cells or connect through Colab MCP (supported clients) or the Colab CLI (macOS or Linux). Check Colab usage limits and save checkpoints outside the runtime.

6 Deliverables and Rules

Each team submits one report and one set of AI session logs on Canvas by the deadline.

1. Challenge submission via AI Coding Gym (6 points, plus up to 3 bonus points). Submit your predictions on the platform by the deadline. You may submit as many times as you like. Your best eligible score will be recorded.

Best eligible MAE m	**Base points**
$m \leq 0.20$	6
$0.20 < m \leq 0.35$	5
$0.35 < m \leq 0.85$	4
$0.85 < m \leq 1.50$	3
$1.50 < m \leq 2.25$	2
$m > 2.25$	1
No eligible score	0

Benchmark	**Best MAE m**	**Bonus points**
Historical median	$m \leq 0.1638$	+1
MLEvolve	$m \leq 0.14237$	+2
Historical bronze	$m \leq 0.1364$	+3

Only the highest qualifying bonus applies.

2. AI interaction history (3 points, required). Every team must upload its complete, original PA3 session logs to Canvas. Follow the course export guide and use CLIcodeLog's **Raw** button. Submit one `.json` or `.jsonl` file, or put multiple original files in `session_log.zip`. Earn 1 point each for completeness, original logs, and correct format. Test the download early. Keep credentials and unrelated conversations out of your PA3 sessions.

3. Final report via Canvas (3 points). Submit a three-to-five-paragraph PDF. Include a screenshot of your leaderboard position. The report should be **mostly human-written**. If you use AI to polish your own text, attach the conversation history for that editing session.

Choose the questions that best show your thinking. You do not need to answer them all.

- **AI inspection summary.** What did the AI agent do? What were its key decisions about models, preprocessing, and features? What weaknesses did you identify in its reasoning trace?
- **Your strategy.** What did you change and why? Describe your approach, the prompts you used with AI assistants, and the domain judgment you applied that the AI did not.
- **Results comparison.** How did your score compare with the AI baseline?
- **Reflection.** Where did the AI fall short? What did you do that it did not? What concept or skill did this teach you?

Data and sources. Use only data downloaded through AI Coding Gym. Do not use the original Kaggle data, mirrors, or datasets or models derived from that data. The challenge's AI Benchmark runs are available for inspection. Do not look for other solutions to this competition or ask an assistant to find them. If your assistant suggests a competition-specific trick unprompted, flag it in your history and report. Credit any outside ideas you use.

7 Building an Agentic ML Loop

An agentic ML loop uses each experiment to decide what to try next. The agent proposes an idea, trains a variant, measures it, and examines the result. A stronger model can become the next baseline. A failed trial can rule out an idea or reveal where the model struggles. You review the evidence and decide which directions are worth pursuing.

To set up the loop, give the agent a baseline it can run, a fixed validation split and metric, and a compute budget. Make sure it can find the training commands and save the code, checkpoints, and trial results. Ask it to record what it tested, how the score and runtime changed, and what it learned. The next experiment should follow from those results.

Choose a goal that fits your project. You might investigate errors for one lung setting, test a feature idea, or improve the model within a GPU budget. Tools that support `/goal` can carry a measurable objective across turns. Give the agent a stopping rule and say what evidence to show when it stops. A regular prompt can describe the same goal and limits.

Tutorials and examples

- Karpathy's autoresearch example uses a fixed evaluation and a log to decide which ML experiments to keep.
- Using Goals in Codex shows how to write a verifiable `/goal` with constraints and a stopping condition.
- Goals in Claude Code explains how `/goal` checks the completion condition across turns.